



Week 2 Solution

Question 1

Question

What are the disadvantages of the best-fit partitioning algorithm?

- a) Might lead to fragmentation
- b) Entire process needs to be in RAM
- c) Limit the size of process by RAM size

Select the most suitable option:

- 1. a, b and c
- 2. b and c
- 3. a and c
- 4. only a

Answer

a, b and c

Solution

The **best-fit** algorithm searches the available memory partitions and selects the smallest partition that is large enough to accommodate the process.

For example, suppose the available holes are:

100 KB
300 KB
500 KB

and a process requires:

280 KB

Best-fit selects the 300 KB hole.

This can lead to several disadvantages.

a) Might lead to fragmentation — True

Best-fit tends to use very small remaining spaces.

For example:

```
Hole = 300 KB  
Process = 280 KB  
Remaining = 20 KB
```

The remaining 20 KB may be too small to be useful for another process.

This contributes to **external fragmentation**.

b) Entire process needs to be in RAM — True

In the traditional contiguous memory allocation model being considered here, a process needs sufficient RAM space for its allocation.

c) Process size is limited by RAM size — True

A process cannot be allocated more memory than the available addressable/main-memory space under this model.

Therefore, all three statements are considered disadvantages.

Correct answer: a, b and c.

Question 2

Question

Consider the following memory map using the multiprogram-with-partition model.

"First fit" searches from left to right.

All sizes have the same unit (kB).

Which memory-request sequence will **not** be satisfied?

1. 50, 80, 80 , following first fit
2. 50, 80, 80 , following best fit
3. 30, 80, 150 , following first fit
4. 30, 80, 150 , following best fit

Answer

30, 80, 150, following first fit

Solution

In **first fit**, the memory is searched from the beginning, and the first available hole large enough for the requested process is selected.

The important point is that the allocation order matters.

For:

30 → 80 → 150

using first fit, the earlier allocations can divide the available holes in such a way that the remaining free spaces are insufficient for the final 150 KB request.

Therefore, even though there may be enough total free memory in different locations, there may not be a **single contiguous hole** large enough for the 150 KB process.

This is an example of the effect of **external fragmentation**.

Hence, the sequence that is not satisfied is:

30, 80, 150, following first fit.

Question 3

Question

What can you **not** find in a process's page table?

1. Present bit
2. Dirty bit
3. Protection bits
4. Sticky bit

Answer

Sticky bit

Solution

A page-table entry contains information required for managing and protecting a virtual page.

Important page-table information can include:

Present bit

Indicates whether the corresponding page is currently present in physical memory.

Present = 1 → page is present

Present = 0 → page is not currently present

Dirty bit

Indicates whether the page has been modified after being loaded into memory.

Protection bits

These specify access permissions, such as whether the page can be:

- Read
- Written
- Executed

Sticky bit

The **sticky bit** is a filesystem/directory permission concept, not a standard process page-table bit.

Therefore, it is the one that cannot be found in the process's page table.

Correct answer: Sticky bit.

Question 4

Question

In a 32-bit processor, what is the maximum process size?

1. 8 GB
2. 2 GB
3. 4 GB
4. 32 GB

Answer

4 GB

Solution

A 32-bit address can represent:

$$2^{32}$$

different addresses.

Since:

$$2^{32} \text{ bytes} = 4 \text{ GB}$$

the theoretical maximum address space is:

$$4 \text{ GB}$$

Calculation:

$$2^{32} = 4,294,967,296 \text{ bytes}$$

Using:

$$1 \text{ GB} = 2^{30} \text{ bytes}$$

we get:

$$\begin{aligned} 2^{32} / 2^{30} \\ &= 2^2 \\ &= 4 \text{ GB} \end{aligned}$$

Therefore, the maximum addressable process size in a 32-bit address space, under the assumptions of the question, is:

4 GB.

Question 5

Question

What is the Dirty bit in the page table used for?

1. Indicates if block is in RAM or not
2. Indicates if contents of RAM are modified with respect to their content in swap space or on disk

Answer

Indicates if contents of RAM are modified with respect to their content in the swap space or on the disk

Solution

The **dirty bit** tells the operating system whether a page has been modified after being loaded into physical memory.

Consider:

```
Disk
|
| Load page
↓
RAM
```

Initially:

```
RAM = Disk
Dirty bit = 0
```

Now suppose the CPU modifies the page:

RAM $\neq$ Disk
Dirty bit = 1

The page is now called **dirty**.

If the operating system needs to remove this page from RAM, it may need to write the modified contents back to disk/swap.

Therefore:

Dirty = 0 $\rightarrow$ Page has not been modified
Dirty = 1 $\rightarrow$ Page has been modified

Hence, the correct answer is option 2.

Question 6

Question

In 2-level page translation, the most significant 10 bits are used to index the page directory.

If the virtual address is 32 bits and each page frame is of size 16 KB, then the number of entries in the page table is:

1. 2^{10}
2. 2^9
3. 2^8
4. 2^7

Answer

2^8

Solution

The virtual address is:

32 bits

The page size is:

16 KB

Convert 16 KB into bytes:

$$\begin{aligned} 16 \text{ KB} &= 16 \times 1024 \\ &= 2^4 \times 2^{10} \\ &= 2^{14} \text{ bytes} \end{aligned}$$

Therefore, the **page offset** requires:

14 bits

The virtual address contains 32 bits.

So the number of bits remaining for the virtual page number is:

$$\begin{aligned} 32 - 14 \\ = 18 \text{ bits} \end{aligned}$$

The question says that the most significant 10 bits are used for the **page directory**.

Therefore, the remaining bits are used for the **page table index**:

$$\begin{aligned} 18 - 10 \\ = 8 \text{ bits} \end{aligned}$$

An 8-bit index can represent:

2^8

entries.

Therefore:

Correct answer: 2^8 .

Question 7

Question

In x86 systems, what converts a logical address to a linear address?

1. CPU
2. Segmentation unit
3. Paging unit
4. Physical memory

Answer

Segmentation unit

Solution

In x86 memory addressing, the logical address is associated with segmentation.

The basic translation is:

Logical Address
↓
Segmentation
↓
Linear Address
↓
Paging
↓
Physical Address

Therefore:

Segmentation

Converts:

Logical address → Linear address

Paging

Converts:

Linear address → Physical address

So the question specifically asks for the component that converts a **logical address to a linear address**.

The answer is:

Segmentation unit.

Question 8

Question

In xv6, which of the following are correct?

Let:

V → Virtual
P → Physical

Options:

1. $V2P(a) = ((\text{uint})(a) - \text{KERNBASE})$
2. $P2V(a) = ((\text{uint})(a) - \text{KERNBASE})$
3. $V2P(a) = ((\text{void} *) (a) + \text{KERNBASE})$
4. $P2V(a) = ((\text{void} *) (a) + \text{KERNBASE})$

Answer

V2P uses subtraction of KERNBASE, and P2V uses addition of KERNBASE.

That is:

$V2P(a) = ((\text{uint})(a) - \text{KERNBASE})$
 $P2V(a) = ((\text{void} *) (a) + \text{KERNBASE})$

Solution

In xv6, the kernel uses a mapping involving `KERNBASE` .

The relationship can be understood as:

Virtual Address = Physical Address + KERNBASE

Therefore, to convert a virtual address back to a physical address:

$$\text{Physical} = \text{Virtual} - \text{KERNBASE}$$

So:

$$\begin{aligned} \text{V2P}(a) \\ = a - \text{KERNBASE} \end{aligned}$$

For the reverse conversion:

$$\text{Virtual} = \text{Physical} + \text{KERNBASE}$$

So:

$$\begin{aligned} \text{P2V}(a) \\ = a + \text{KERNBASE} \end{aligned}$$

Therefore, the correct pair is:

$$\begin{aligned} \text{V2P}(a) &\rightarrow \text{subtract KERNBASE} \\ \text{P2V}(a) &\rightarrow \text{add KERNBASE} \end{aligned}$$

Correct answer: Options 1 and 4.

Question 9

Question

Which function in xv6 removes free pages from the head of the linked list?

1. `kfree()`
2. `kalloc()`
3. `malloc()`

4. `free()`

Answer

`kalloc()`

Solution

xv6 maintains a linked list of free physical pages.

When the kernel needs a free physical page, it uses:

```
kalloc()
```

`kalloc()` takes a free page from the free-page list.

Conceptually:

Free page list:

[Page A] → [Page B] → [Page C] → NULL

↑

head

`kalloc()`

↓

[Page B] → [Page C] → NULL

The first page is removed from the free list and returned to the caller.

On the other hand:

```
kfree()
```

is used to return a page to the free list.

Therefore:

Correct answer: `kalloc()`

Question 10

Question

When a CPU boots, which of the following registers are **not set to zero**?

Multiple options might be correct.

1. Code segment register
2. Instruction pointer
3. Stack pointer
4. Base pointer

Answer

Code segment register and Instruction pointer

Solution

At CPU startup, the processor begins execution from a predefined initial state rather than simply treating every register as zero.

The important registers in this question are:

Code Segment Register

The code segment register is initialized with a predefined value associated with the CPU's startup state.

Therefore, it is not simply zero.

Instruction Pointer

The instruction pointer is also initialized to a predefined startup address/state so that the

processor knows where execution begins.

Therefore, it is also not zero.

The assignment identifies these two as the registers that are not set to zero:

Code segment register

Instruction pointer